Final Exam Project Proposal

LibraTrack

University Library Management System

Discipline	Development of Web Applications in Java Spring
Work format	Individual (1 student)
Student	Gabit Mukan
Group	[Group]
Instructor	[Instructor name]
UX/UI design (prototype)	https://gabit399.github.io/libratrack/docs/prototype/
GitHub repository	https://github.com/Gabit399/libratrack
Date	October 31, 2026

1. Project Description

1.1 Project title

LibraTrack — University Library Management System

1.2 Project description

LibraTrack is a web-based library management system for a university library, built with Java Spring Boot. Many campus libraries still rely on paper cards and spreadsheets: students cannot check whether a book is available without visiting the library, librarians record every loan by hand, and overdue books are difficult to track. LibraTrack moves this whole process online.

Readers register with their university email, search the catalog by title, author, ISBN or genre, see how many copies of each book are currently available and reserve a copy for pickup. A personal “My loans” page shows active reservations, borrowed books, due dates and overdue items, and allows the reader to extend a loan once. Librarians work in a separate dashboard where they add, edit and remove books, manage the number of copies, issue reserved books, register returns and monitor statistics such as active and overdue loans.

The application follows the layered Spring architecture (Controller → Service → Repository) and stores its data in a PostgreSQL database through Spring Data JPA. Pages are rendered on the server with Thymeleaf, access is protected by Spring Security with two roles (READER and LIBRARIAN), and all user input is validated with Spring Validation. The project is developed individually by one student using Git and GitHub.

1.3 Main objectives

1. Provide an online catalog with search and filtering that shows real-time book availability.
2. Digitise the reservation → issue → return lifecycle of a book loan.
3. Give readers a personal space to track loans, due dates and overdue books.
4. Give librarians tools to manage the book collection and loans in one place.
5. Apply a clean layered Spring Boot architecture with a relational database, validation, security and tests.

1.4 Target users

User	Needs
Reader (students, teachers)	Find books quickly, know whether a copy is available, reserve it online, see due dates and avoid fines.
Librarian (administrator)	Maintain the catalog, process reservations, register issues and returns, see overdue loans and basic statistics.

1.5 Problem the application solves

- No way to check book availability remotely — readers waste time visiting the library for books that are already on loan.
- Manual paper or spreadsheet records are slow, error-prone and easy to lose.
- Due dates are not tracked automatically, so overdue books are noticed late.
- Librarians have no quick overview of how the collection is used.

1.6 Functional requirements

ID	Requirement	Role
FR-1	The system shall allow a user to register with full name, email, student ID and password, and to log in / log out.	Guest
FR-2	The system shall display the book catalog with search by title, author or ISBN and filters by genre and availability.	Reader
FR-3	The system shall show a book details page with description, ISBN, shelf location and number of available copies.	Reader
FR-4	The system shall allow a reader to reserve an available book; the number of available copies decreases automatically.	Reader
FR-5	The system shall show the reader’s loans (reserved, borrowed, overdue, returned) with due dates, and allow cancelling a reservation or extending a loan once by 7 days.	Reader
FR-6	The system shall allow a librarian to create, update and delete books (CRUD) with server-side validation.	Librarian
FR-7	The system shall allow a librarian to issue reserved books and register returns.	Librarian
FR-8	The system shall mark loans as overdue after the due date and show statistics (books, active loans, overdue loans, readers) on the librarian dashboard.	Librarian
FR-9	The system shall restrict pages by role: readers cannot access librarian pages.	All

1.7 Expected results

- A working Spring Boot web application with at least 8 pages (see UX/UI design) connected to a PostgreSQL database.
- Full reservation flow: login → search → book details → reserve → “My loans”.
- Librarian dashboard with book CRUD and loan processing.
- Role-based security, validated forms and clear error messages.
- Unit tests for the service layer (JUnit 5, Mockito) and a Postman collection for the REST endpoints.
- Source code in a public GitHub repository with README and setup instructions.

2. UX/UI Design

The interface is designed as a clickable prototype with **8 screens**. Link: **https://gabit399.github.io/libratrack/docs/prototype/**.

#	Screen	Purpose / main elements
1	Login	Email + password form, link to registration, demo entry for librarian.
2	Registration	Full name, email, student ID, password; validation messages.
3	Catalog	Search bar, genre and availability filters, grid of book cards with availability badges.
4	Book details	Cover, description, ISBN, shelf, copies available; “Reserve” button with confirmation dialog.
5	My loans	Table of reservations and loans with status badges; cancel / extend actions.
6	Profile	Edit personal data, log out.
7	Librarian dashboard	Statistic cards, pending reservations table with “Issue” action.
8	Add / edit book	Book form (title, author, ISBN, copies, genre, description) with validation.

Design system

Colors

- Primary `#2F5D50`
- Accent `#D98B3A`
- Background `#F6F5F1`
- Success `#2F7D4F` Danger `#B4443A`

Typography & components

Font: Inter / system sans-serif. H1 26px bold, H2 19px, body 15px.

Shared components: top navigation bar, primary / secondary buttons, input fields, book card, status badge, data table, confirmation modal, toast notification.

Main user flow — reserving a book

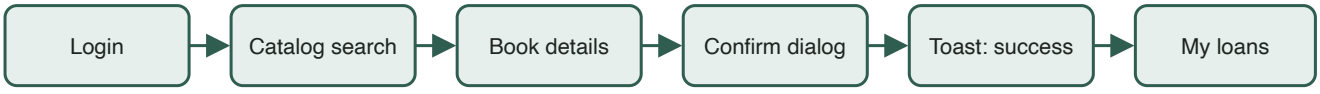


Figure 1. Complete user flow implemented in the clickable prototype

Navigation: readers move between Catalog, My loans and Profile through the top bar; librarians between Dashboard and Add book. Every screen links back to its parent screen.

3. Technology Stack and Dependencies

Component	Technology	Why we chose it

Programming language	Java 21 (LTS)	Required by the course; LTS version supported by Spring Boot 3.
Backend framework	Spring Boot 3.x	Auto-configuration, embedded Tomcat, starters — fast setup of a production-style app.
Web layer	Spring MVC / Spring Web	Controllers for HTML pages and REST endpoints.
Frontend	Thymeleaf, HTML5, CSS3, vanilla JavaScript	Server-side rendering integrates natively with Spring MVC and Spring Security; JS `fetch` is used for live search.
Database	PostgreSQL 16	Reliable open-source relational DB; our data (users, books, loans) is relational.
Persistence	Spring Data JPA / Hibernate	Maps entities to tables; repositories give CRUD and query methods without boilerplate SQL.
Build tool	Maven	Standard dependency management, generated by Spring Initializr.
Version control	Git / GitHub	Version history, feature branches and a public repository for review.
Testing	JUnit 5, Mockito, Postman	Unit tests of services; manual and collection-based testing of REST endpoints.

Spring Boot dependencies

Dependency (artifactId)	Why it is necessary
`spring-boot-starter-web`	Spring MVC, embedded Tomcat, JSON (Jackson) — handles HTTP requests and REST API.
`spring-boot-starter-data-jpa`	Hibernate + Spring Data repositories for the Book, User and Loan entities.
`spring-boot-starter-thymeleaf`	Renders HTML templates for all pages.
`postgresql` (driver)	JDBC driver to connect to the PostgreSQL database.
`lombok`	Generates getters, setters, constructors and builders — less boilerplate in entities and DTOs.
`spring-boot-starter-validation`	Bean Validation (`@NotBlank` , `@Email` , `@Size` , `@Min`) for registration and book forms.
`spring-boot-starter-security`	Login/logout, BCrypt password hashing, role-based access (READER / LIBRARIAN).
`thymeleaf-extras-springsecurity6`	Show or hide UI elements in templates by role (`sec:authorize`).
`spring-boot-starter-test`	JUnit 5, Mockito, AssertJ and MockMvc for unit and controller tests.
`spring-boot-devtools` (dev only)	Automatic restart and live reload during development.

4. Team Members and Responsibilities

The project is completed **individually**, so one student is responsible for all project components: frontend, backend, database and testing.

Student	Area	Responsibilities
Gabit Mukan	UX/UI Designer	Figma design and clickable prototype; design system (colors, typography, components).
	Backend Developer	Spring Boot project setup; controllers, services, REST API (`/api/books`, `/api/loans`); Spring Security and roles; business rules (reservation, extension, overdue).
	Database Developer	ER model; JPA entities and relationships; repositories and custom queries; seed data.
	Frontend & QA	Thymeleaf templates, CSS, JavaScript live search; integration; JUnit/Mockito tests and Postman collection; GitHub repository.

The student understands the whole application architecture and presents all parts of the project during the Final Exam presentation.

5. System Architecture

LibraTrack follows the layered Spring architecture: **Frontend** → **Controller** → **Service** → **Repository** → **Database**.

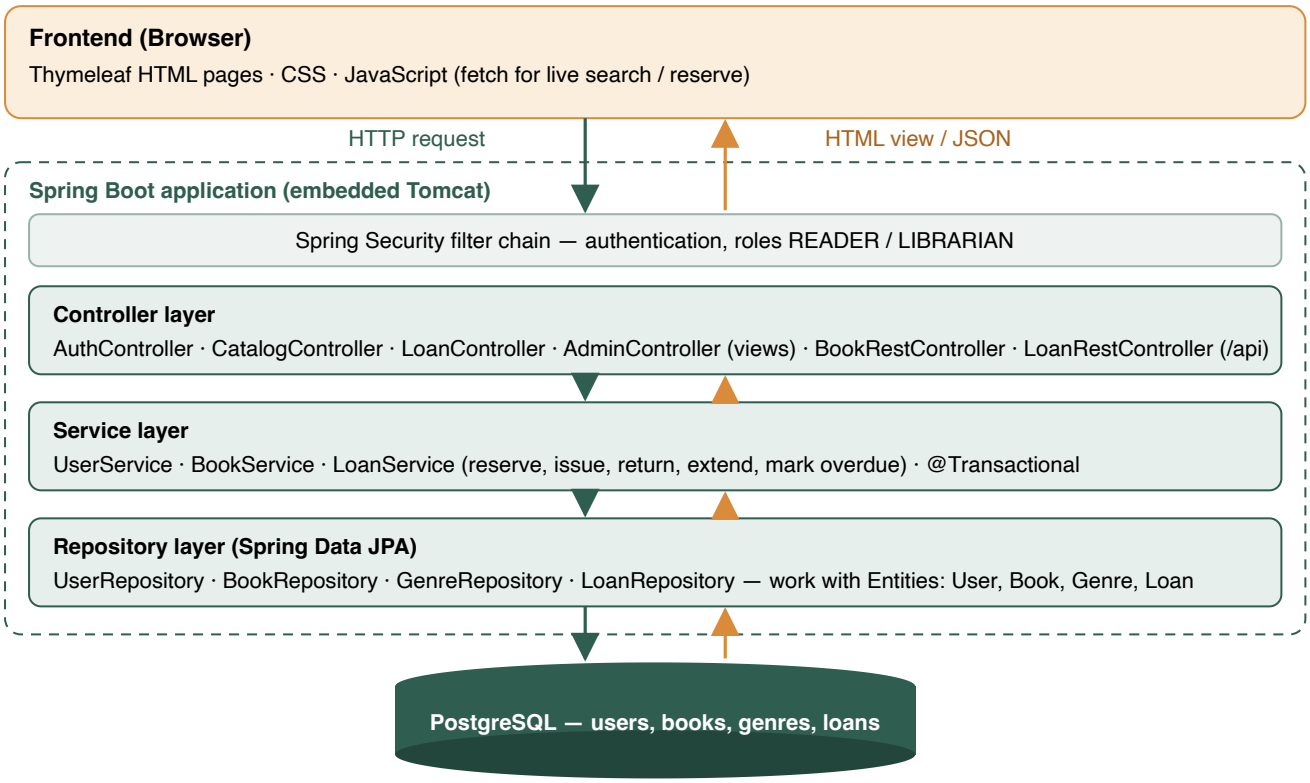


Figure 2. Layered architecture and data flow (green — request, orange — response)

Responsibilities of the layers

Layer	Responsibility in LibraTrack
Controller	Processes HTTP requests and returns views or JSON. Example: `CatalogController` handles `GET /books` and returns the `catalog.html` view; `LoanRestController` handles `POST /api/loans` and returns a `LoanDto`. Validates input with `@Valid`, never contains business logic.
Service	Implements business logic: a book can be reserved only if available copies > 0; a reader can have at most 5 active loans; a loan can be extended once; loans past the due date become OVERDUE (scheduled check with `@Scheduled`). Methods are `@Transactional`.
Repository	Handles database access via `JpaRepository` interfaces, e.g. `findByTitleContainingIgnoreCaseOrAuthorContainingIgnoreCase(...)`, `findByUserIdOrderByReservedAtDesc(...)`, `countByStatus(...)`.
Entity / Model	Represents application data: `User`, `Book`, `Genre`, `Loan`; DTOs and form objects transfer data between layers.
Database	PostgreSQL stores and manages all information; schema generated by Hibernate (`ddl-auto=update`) during development.

Data model (entities)

Entity	Fields	Relationships
User	id, fullName, email (unique), studentId, password (BCrypt), role (READER / LIBRARIAN), createdAt	One User → many Loans
Book	id, title, author, isbn (unique), description, totalCopies, availableCopies, shelfLocation, coverUrl	Many Books → one Genre; one Book → many Loans
Genre	id, name	One Genre → many Books
Loan	id, reservedAt, issuedAt, dueDate, returnedAt, extended (boolean), status (RESERVED, BORROWED, OVERDUE, RETURNED, CANCELLED)	Many Loans → one User; many Loans → one Book

Frontend ↔ backend communication

Most pages are rendered on the server: the browser sends a request, the controller puts data into the `Model` and returns a Thymeleaf template. For interactive parts the page's JavaScript calls REST endpoints with `fetch()` and exchanges JSON; Spring Security protects these endpoints using the same session and a CSRF token.

Method	Endpoint	Description	Access
GET	`/api/books?q=&genre=&available=`	Search books (live search on catalog)	Reader
GET	`/api/books/{id}`	Book details	Reader
POST	`/api/books`	Create a book	Librarian
PUT / DELETE	`/api/books/{id}`	Update / delete a book	Librarian
POST	`/api/loans`	Reserve a book (`{ "bookId": 5 }`)	Reader
GET	`/api/loans/my`	Current user's loans	Reader
PATCH	`/api/loans/{id}/extend` · `/cancel`	Extend or cancel a loan	Reader
PATCH	`/api/loans/{id}/issue` · `/return`	Issue a reserved book / register return	Librarian

6. Development Roadmap

Stage	Description	Main tasks	Responsible	Week
1	Project requirements and planning	Functional requirements, user roles, proposal document	Gabit Mukan	1
2	UX/UI design	Figma screens, design system, clickable prototype	Gabit Mukan	1–2

3	Spring Boot project setup	Spring Initializr, dependencies, package structure, GitHub repo, `application.yml`	Gabit Mukan	2
4	Database and entity development	ER model, JPA entities, relationships, seed data	Gabit Mukan	3
5	Repository, Service, Controller implementation	Repositories, business logic, MVC + REST controllers, Spring Security	Gabit Mukan	3–4
6	Frontend development	Thymeleaf templates, CSS, JavaScript, validation messages	Gabit Mukan	4–5
7	Integration and testing	Connect UI with backend, JUnit/Mockito tests, Postman collection, bug fixing	Gabit Mukan	5–6
8	Final Exam presentation and demonstration	Slides, demo scenario, README	Gabit Mukan	7

7. Submission Links

UX/UI design (≥ 5 screens)	https://gabit399.github.io/libratrack/docs/prototype/
GitHub repository	https://github.com/Gabit399/libratrack